Cryptography and Network Security (ICS412)

Course Project Report

Secure Chat Application using Diffie-Hellman Key Exchange, AES-256-CBC, and HMAC-SHA256

With Real-Time Network Attack Simulation and Dynamic Adversarial Workbench

---

Submitted By:

Name	Roll Number	Program
Vishal Rawat	2023BCS0101	B.Tech Computer Science & Engineering
Jatin Nath	2023BCD0014	B.Tech CSE (Data Science)
Rishi	2023BCD0023	B.Tech CSE (Data Science)

Department of Computer Science and Engineering

October 2026

Abstract

In contemporary computer networks, public communication channels are fundamentally untrusted and inherently susceptible to interception, manipulation, and spoofing. This project presents the design, mathematical formalization, and interactive implementation of an end-to-end secure communication system modeled after modern transport layer security protocols (such as TLS 1.3 and the Signal Protocol architecture). The system integrates four foundational pillars of modern applied cryptography:

1. **Asymmetric Key Exchange:** Diffie-Hellman protocol based on RFC 3526 2048-bit MODP safe prime groups (p, g) to establish a shared master secret over an insecure channel without transmitting private exponents.
2. **Key Derivation Function (KDF):** HMAC-based Extract-and-Expand Key Derivation Function (HKDF-SHA256, RFC 5869) to derive independent, cryptographically isolated 256-bit symmetric encryption keys (K_{AES}) and message authentication keys (K_{HMAC}).
3. **Symmetric Payload Confidentiality:** Advanced Encryption Standard (AES-256) operating in Cipher Block Chaining (CBC) mode with cryptographically secure random Initialization Vectors (IV, 128-bit) per message and PKCS#7 padding.
4. **Message Integrity, Authentication, and Anti-Replay Defense:** Encrypt-then-MAC (EtM) paradigm utilizing HMAC-SHA256 computed over the 64-bit monotonically increasing sequence number, IV, and ciphertext payload (Seq $\|$ IV $\|$ C).

To provide rigorous educational insight and empirical evaluation, the application embeds a real-time, tri-party visual environment containing Alice (sender), Bob (receiver), and Eve (network adversary). Through this platform, four critical adversarial scenarios are demonstrated: passive eavesdropping, ciphertext tampering (single-bit flipping), packet replay attacks, and unauthenticated Diffie-Hellman Man-in-the-Middle (MITM) interception. The system has been validated both as a standalone Python application leveraging Python `cryptography` primitives and as a zero-dependency web interface utilizing the W3C Web Cryptography API (`crypto.subtle`).

Contents

1 Introduction **1**
1.1 Motivation and Problem Context . . . 1
1.2 The Cryptographic Dilemma . . . 1
1.3 Proposed Hybrid Architecture . . . 1

2 System Architecture & Threat Model **2**
2.1 Tri-Party System Architecture . . . 2
2.2 Adversary Capabilities (Dolev-Yao Threat Model) . . . 2

3 Cryptographic Protocols & Mathematical Formulations **2**
3.1 Phase I: Diffie-Hellman Key Exchange (RFC 3526 Group 14) . . . 2
3.2 Phase II: Key Derivation Function (HKDF-SHA256, RFC 5869) . . . 3
3.3 Phase III: AES-256-CBC Payload Confidentiality . . . 3
3.4 Phase IV: Encrypt-then-MAC (EtM) Authentication . . . 4
3.5 Phase V: Receiver Decryption & Anti-Replay Verification Pipeline . . . 4

4 Attack Simulation Mechanisms & Defense Analysis **4**

5 Experimental Results & Screenshot Evaluation **5**
5.1 Figure 1: Initial Application State (Channel Not Secured) . . . 5
5.2 Figure 2: Step-by-Step Diffie-Hellman Key Exchange & HKDF Derivation . . . 6
5.3 Figure 3: Secure End-to-End Chat & Eavesdropper View . . . 6

5.4 Figure 4: Active Ciphertext Tampering Attack (Rejected by HMAC) 7
5.5 Figure 5: Replay Attack (Rejected by Anti-Replay Counter) 8
5.6 Figure 6: Unauthenticated DH Man-in-the-Middle (MITM) Interception 9
5.7 Figure 7: MITM Traffic Eavesdropping and Payload Manipulation 10

6 Implementation Code **10**

7 Security Discussion & Theoretical Guarantees **11**
7.1 Why Encrypt-then-MAC (EtM)? 11
7.2 Mitigating Diffie-Hellman MITM Attacks 12

8 Conclusion **12**

References **12**

1 Introduction

1.1 Motivation and Problem Context

The rapid proliferation of networked applications, distributed computing systems, and real-time messaging services has elevated information security from an operational feature to a critical architectural requirement. Open packet-switched networks (such as the Internet) inherently expose packets to inspection and unauthorized modification by intermediate entities, including untrusted routers, ISP taps, and active adversaries.

Achieving secure communication over an untrusted transmission medium necessitates fulfilling four fundamental security objectives defined by the CIA triad and extended cryptographic principles:

- **Confidentiality:** Preventing unauthorized disclosure of plaintext data to passive observers.
- **Integrity:** Ensuring that any modification, insertion, or deletion of transmitted data in transit is reliably detected by the recipient.
- **Authenticity & Non-Repudiation:** Confirming that the received payload originates genuinely from the claimed peer entity.
- **Freshness & Anti-Replay Defense:** Guaranteeing that previously intercepted valid packets cannot be maliciously retransmitted to induce state duplication or transaction replay.

1.2 The Cryptographic Dilemma

Symmetric ciphers such as AES-256 provide high-throughput, computationally lightweight encryption suitable for bulk data transfer. However, symmetric ciphers suffer from the fundamental **Key Distribution Problem**: two parties wishing to communicate must securely share a secret key beforehand. Transmitting the key across the open channel nullifies confidentiality.

Conversely, asymmetric (public-key) algorithms solve the key distribution dilemma through mathematically related key pairs but incur substantial computational overhead, rendering them impractical for streaming message encryption.

1.3 Proposed Hybrid Architecture

This project synthesizes asymmetric and symmetric paradigms into a **hybrid cryptographic architecture**:

1. **Diffie-Hellman (DH)** asymmetric key agreement establishes an ephemeral shared secret without disclosing the participants' private exponents.
2. **HKDF-SHA256** stretches and separates the shared entropy into dedicated operational keys, eliminating cryptographic key-reuse vulnerabilities.
3. **AES-256-CBC** secures arbitrary-length plaintexts using fresh initialization vectors.
4. **HMAC-SHA256 (Encrypt-then-MAC)** binds message integrity and sequence counter tracking to the derived MAC key, thwarting chosen-ciphertext attacks, padding oracle vulnerabilities, and replay attacks.
5. **Interactive Attack Workbench:** Eve's adversarial tap provides visibility into raw network packets, verifying that an adversary observes only pseudo-random bitstrings while interactively demonstrating failure modes when packets are corrupted or replayed.

2 System Architecture & Threat Model

2.1 Tri-Party System Architecture

The system architecture consists of two legitimate communication endpoints (**Alice** and **Bob**) communicating across a shared, untrusted network tap controlled or observed by an adversary (**Eve**).

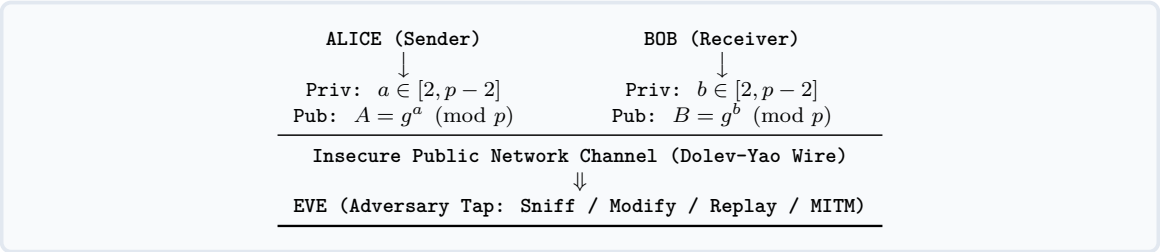


Figure 1: Conceptual tri-party network topology with active/passive adversary tap.

2.2 Adversary Capabilities (Dolev-Yao Threat Model)

We formalize Eve's capabilities under the canonical **Dolev-Yao threat model**:

- **Passive Eavesdropping:** Eve reads all raw bits transmitted across the wire (A, B, Seq, IV, C, and Tag).
- **Active Message Modification:** Eve can intercept and alter arbitrary bits of transmitted packets in transit before they reach the recipient.
- **Message Replay:** Eve can capture legitimate packets transmitted earlier and re-inject them into the network channel.
- **Man-in-the-Middle (MITM):** In the absence of digital signatures or public key infrastructure (PKI), Eve can intercept the public Diffie-Hellman values A and B, replace them with her own parameters Y_{E_1}, Y_{E_2}, and establish distinct shared keys with Alice and Bob separately.

3 Cryptographic Protocols & Mathematical Formulations

3.1 Phase I: Diffie-Hellman Key Exchange (RFC 3526 Group 14)

The Diffie-Hellman protocol operates over the multiplicative group of integers modulo a large safe prime p, where $p = 2q + 1$ (with q prime) and g is a generator of order q. In our implementation, we adopt RFC 3526 MODP Group 14 (2048-bit prime):

$$\text{Group Parameters: } p \text{ (2048-bit safe prime)}, \quad g = 2 \tag{1}$$

1. **Private Exponent Selection:**

$$\text{Alice chooses: } a \xleftarrow{\mathcal{R}} [2, p-2] \tag{2}$$

$$\text{Bob chooses: } b \xleftarrow{\mathcal{R}} [2, p-2] \tag{3}$$

2. **Public Key Computation:**

$$A = g^a \pmod p, \qquad B = g^b \pmod p \tag{4}$$

3. **Public Key Transmission:** Alice transmits A to Bob; Bob transmits B to Alice across the insecure channel.

4. **Shared Secret Agreement:**

$$\text{Alice computes: } K_{\text{shared}} = B^a \pmod p = (g^b)^a \pmod p = g^{ab} \pmod p \tag{5}$$
$$\text{Bob computes: } K_{\text{shared}} = A^b \pmod p = (g^a)^b \pmod p = g^{ab} \pmod p \tag{6}$$

Due to the **Discrete Logarithm Problem (DLP)** and the **Computational Diffie-Hellman (CDH)** assumption, computing $g^{ab} \pmod p$ given (p, g, A, B) is computationally intractable for 2048-bit modulus size within polynomial time.

3.2 Phase II: Key Derivation Function (HKDF-SHA256, RFC 5869)

The raw shared secret K_{shared} from Diffie-Hellman is not uniformly distributed across its bit length. Using it directly as an AES key introduces potential key-schedule weaknesses. We employ HKDF-SHA256 (RFC 5869):

$$\text{PRK} = \text{HMAC-SHA256}(\text{Salt} = \text{None}, K_{\text{shared}}) \tag{7}$$
$$\text{OKM} = \text{HKDF-Expand}(\text{PRK}, \text{info} = \text{"secure-chat-lab"}, L = 64 \text{ bytes}) \tag{8}$$

The 64-byte Output Keying Material (OKM) is partitioned into two distinct cryptographic keys:

$$K_{\text{AES}} = \text{OKM}[0 \ldots 31] \quad \text{(256-bit symmetric encryption key)} \tag{9}$$
$$K_{\text{HMAC}} = \text{OKM}[32 \ldots 63] \quad \text{(256-bit message authentication key)} \tag{10}$$

This strict key separation prevents cross-protocol cryptographic interactions and key reuse vulnerabilities.

3.3 Phase III: AES-256-CBC Payload Confidentiality

To encrypt an arbitrary plaintext message M:

1. **Sequence Number Tracking:** Increment the local 64-bit integer counter:

$$\text{Seq} \leftarrow \text{Seq} + 1 \tag{11}$$

2. **Initialization Vector (IV) Generation:** A fresh 128-bit random IV is generated using a cryptographically secure pseudo-random number generator (CSPRNG):

$$\text{IV} \xleftarrow{\mathcal{R}} \{0,1\}^{128} \tag{12}$$

3. **PKCS#7 Padding:** Plaintext bytes P are padded to a multiple of the 16-byte block size:

$$\text{PadLength} = 16 - (|P| \bmod 16) \tag{13}$$

where each pad byte is filled with the integer value PadLength.

4. **CBC Block Cipher Chaining:** For padded blocks $P_1, P_2, \ldots, P_k$:

$$C_0 = \text{IV} \tag{14}$$
$$C_i = \text{AES-256}_{K_{\text{AES}}}(P_i \oplus C_{i-1}), \quad \forall i \in \{1, 2, \ldots, k\} \tag{15}$$
$$C = C_1 \parallel C_2 \parallel \cdots \parallel C_k \tag{16}$$

3.4 Phase IV: Encrypt-then-MAC (EtM) Authentication

Cryptographic consensus (Krawczyk, 2001; Bellare & Namprempre, 2000) proves that **Encrypt-then-MAC** provides IND-CCA2 security and prevents padding oracle attacks:

$$T = \text{HMAC-SHA256}_{K_{\text{HMAC}}}\left(\text{Seq}_{64} \parallel \text{IV}_{128} \parallel C\right) \tag{17}$$

The serialized binary packet transmitted across the network is structured as:

$$\text{Packet} = \underbrace{\text{Seq}}_{8\text{ bytes}} \parallel \underbrace{\text{IV}}_{16\text{ bytes}} \parallel \underbrace{C}_{N\times 16\text{ bytes}} \parallel \underbrace{T}_{32\text{ bytes}} \tag{18}$$

Seq (8 Bytes)	IV (16 Bytes)	Ciphertext ($N \times 16$ Bytes)	HMAC Tag (32 Bytes)
Monotonic Counter	Random CSPRNG	AES-256-CBC Encrypted	HMAC-SHA256 (EtM)

Figure 2: Wire packet layout ensuring confidentiality, authenticity, integrity, and freshness.

3.5 Phase V: Receiver Decryption & Anti-Replay Verification Pipeline

Upon receipt of a network packet, the receiver executes a strict, defense-in-depth pipeline:

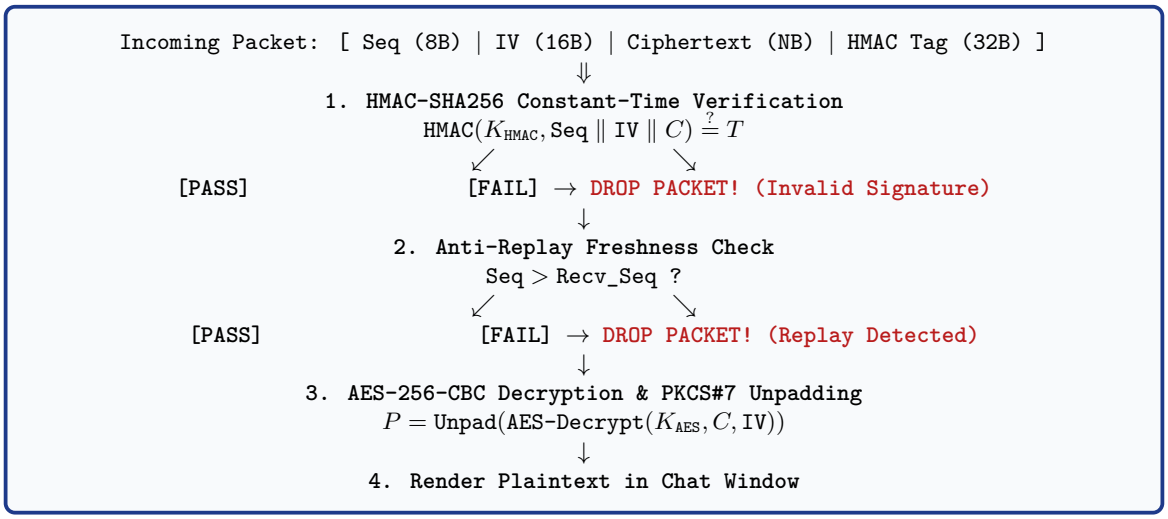


Figure 3: Receiver verification pipeline. Integrity verification strictly precedes decryption.

Core Security Guarantee: Integrity Before Decryption

Decryption is **never attempted** on untrusted ciphertext. Because the HMAC tag is verified in constant time first, any modified ciphertext or tampered byte is rejected immediately without parsing PKCS#7 padding, mathematically neutralizing padding oracle attacks.

4 Attack Simulation Mechanisms & Defense Analysis

The system incorporates interactive attack toggles enabling empirical validation of adversarial scenarios:

Attack Scenario	Adversarial Mechanism	System Defense & Detection
Passive Eavesdropping	Eve sniffs wire packets capturing $\text{Seq}, \text{IV}, C, T$.	Ciphertext exhibits maximum entropy. Without K_{AES}, decryption is computationally infeasible.
Active Ciphertext Tampering	Eve flips a single bit in the ciphertext: $C[2] \leftarrow C[2] \oplus 0\text{x}01$.	HMAC-SHA256 verification fails instantly. Packet dropped with explicit visual alert; decryption is aborted.
Packet Replay Attack	Eve re-transmits a previously captured valid packet.	Sequence check fails ($\text{Seq} \leq \text{Recv_Seq}$). Packet dropped as duplicate/replay.
Man-in-the-Middle (MITM)	Eve intercepts public parameters A, B and substitutes Y_{E_1}, Y_{E_2}.	Demonstrates the vulnerability of unauthenticated DH. Highlights the requirement for X.509 certificates and PKI signatures.

Table 1: Comprehensive breakdown of simulated attacks and corresponding defensive primitives.

5 Experimental Results & Screenshot Evaluation

The application was empirically tested through both the Python Tkinter interface and the Web Cryptography interface. Below are test captures documenting each operational phase.

5.1 Figure 1: Initial Application State (Channel Not Secured)

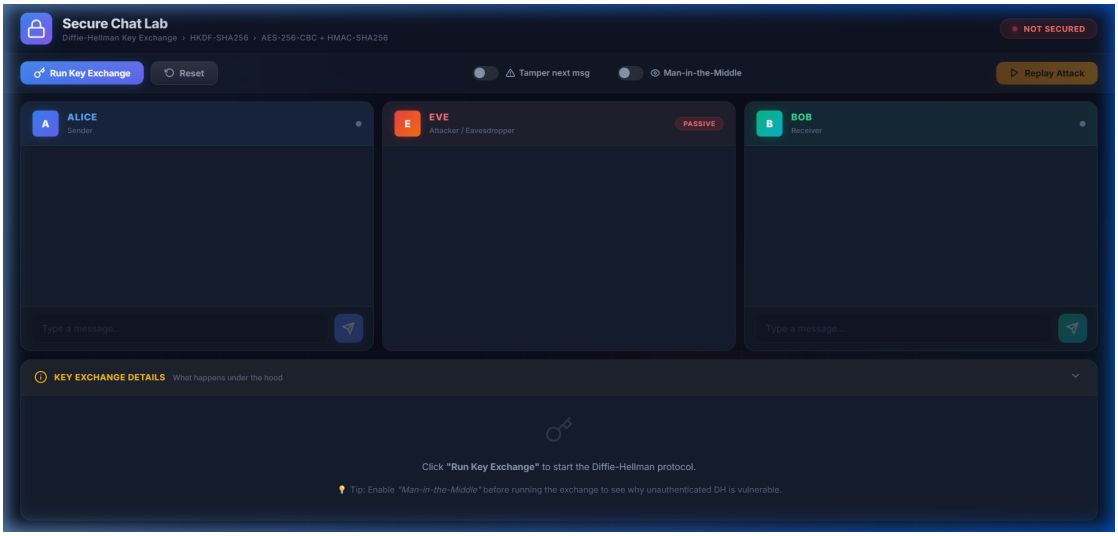


Figure 4: Application interface at startup prior to cryptographic handshake.

Analysis: The interface displays Alice (blue, left), Bob (green, right), and Eve (red, center). Chat inputs remain locked until secure parameters are established. The status badge indicates `NOT SECURED`.

5.2 Figure 2: Step-by-Step Diffie-Hellman Key Exchange & HKDF Derivation

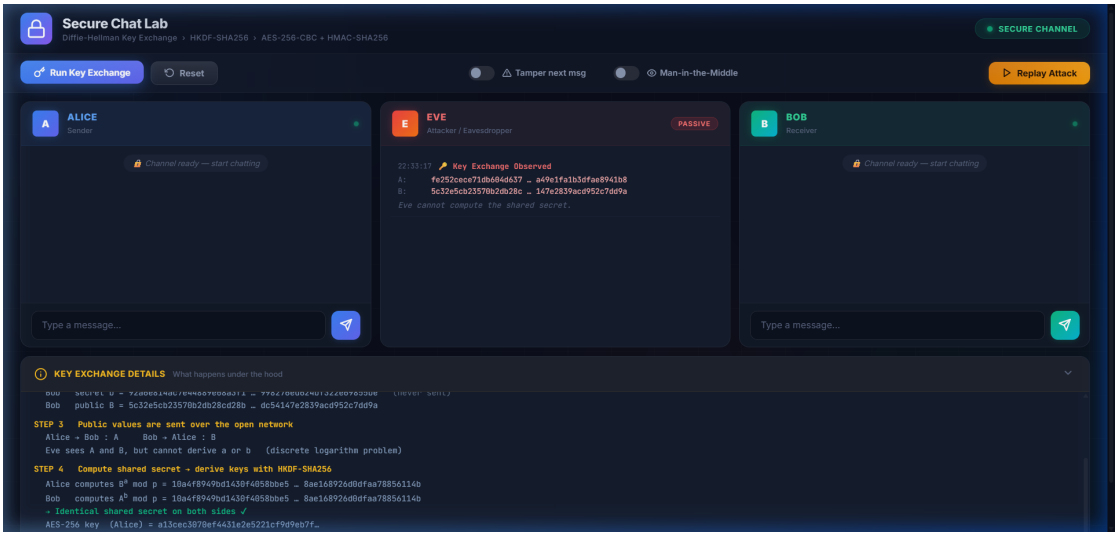


Figure 5: Execution of RFC 3526 DH Key Exchange and HKDF-SHA256 key expansion.

Analysis: The lower telemetry log details the 2048-bit prime p, generator $g = 2$, private exponents a and b, computed public values A and B, shared secret $g^{ab} \pmod{p}$, and derived 256-bit AES and HMAC keys. The channel transitions to `SECURE CHANNEL`.

5.3 Figure 3: Secure End-to-End Chat & Eavesdropper View

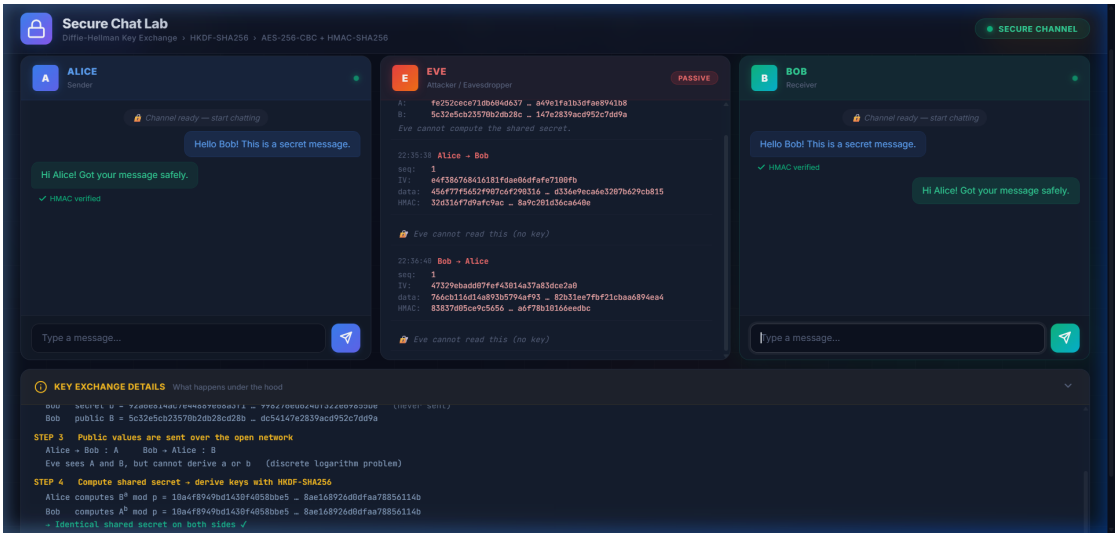


Figure 6: Confidential message exchange with eavesdropper wire view.

Analysis: Plaintext messages are displayed with green HMAC verification badges on legitimate endpoints. Meanwhile, Eve captures only the sequence counter, random IV, high-entropy AES ciphertext, and HMAC tag. Eve's log indicates: *"Eve cannot read this (no key)"*.

5.4 Figure 4: Active Ciphertext Tampering Attack (Rejected by HMAC)

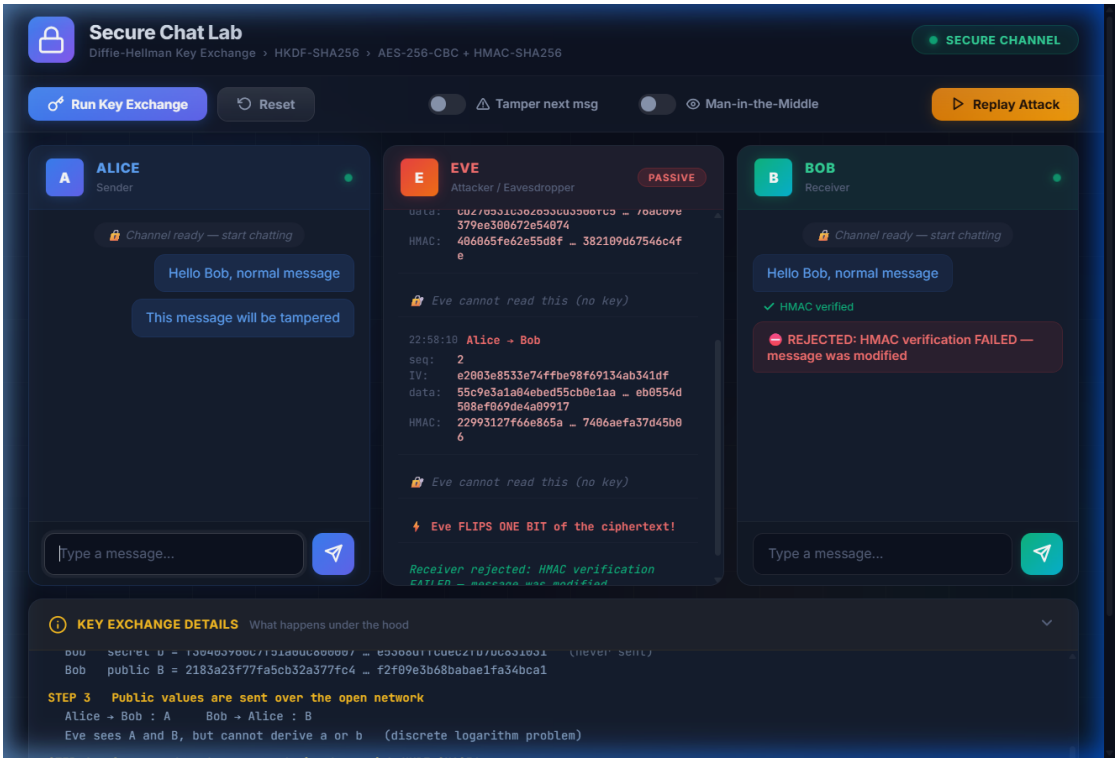


Figure 7: Single-bit tampering detected and rejected by HMAC verification.

Analysis: Eve flips bit 0 of byte 26 in transit. Bob recomputes HMAC-SHA256 over the wire payload; verification fails immediately. Bob displays `REJECTED: HMAC verification FAILED - message was modified`, completely preventing corrupted payload decryption.

5.5 Figure 5: Replay Attack (Rejected by Anti-Replay Counter)

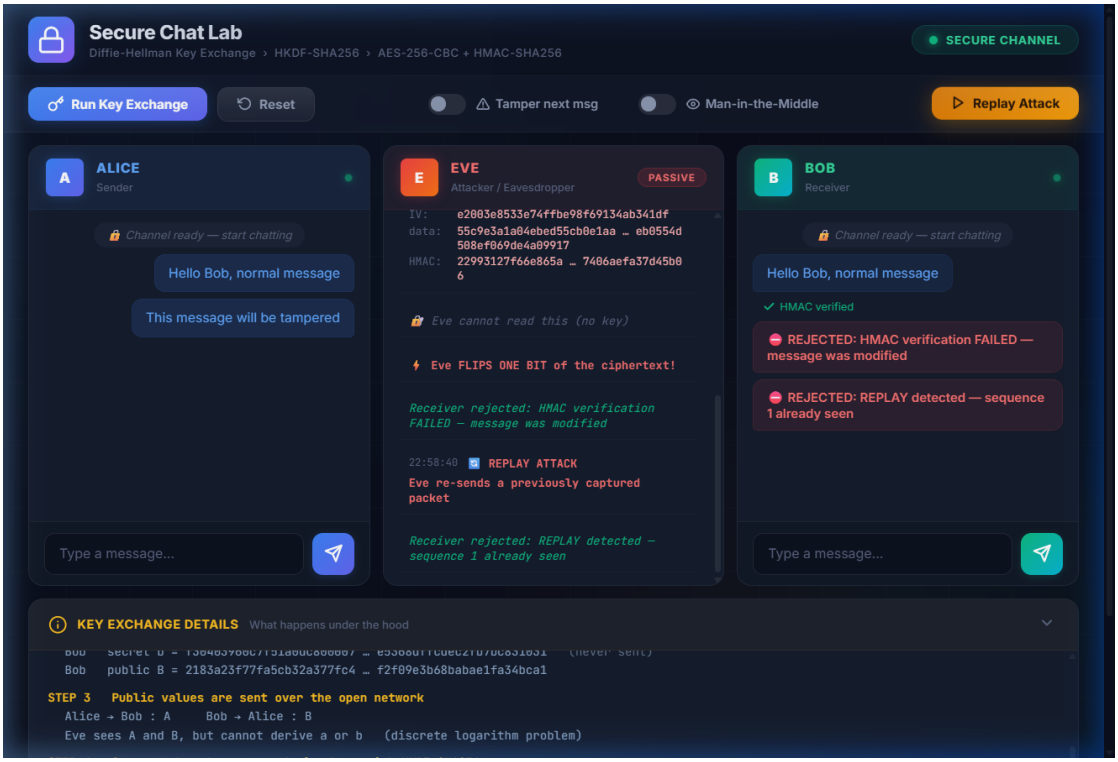


Figure 8: Replay attack blocked by monotonic sequence counter verification.

Analysis: Eve re-injects a previously valid packet into the wire. Bob's HMAC verification passes, but the freshness check determines that Seq $\leq$ Recv_Seq. The packet is dropped with: `REJECTED: REPLAY detected - sequence 1 already seen`.

5.6 Figure 6: Unauthenticated DH Man-in-the-Middle (MITM) Interception

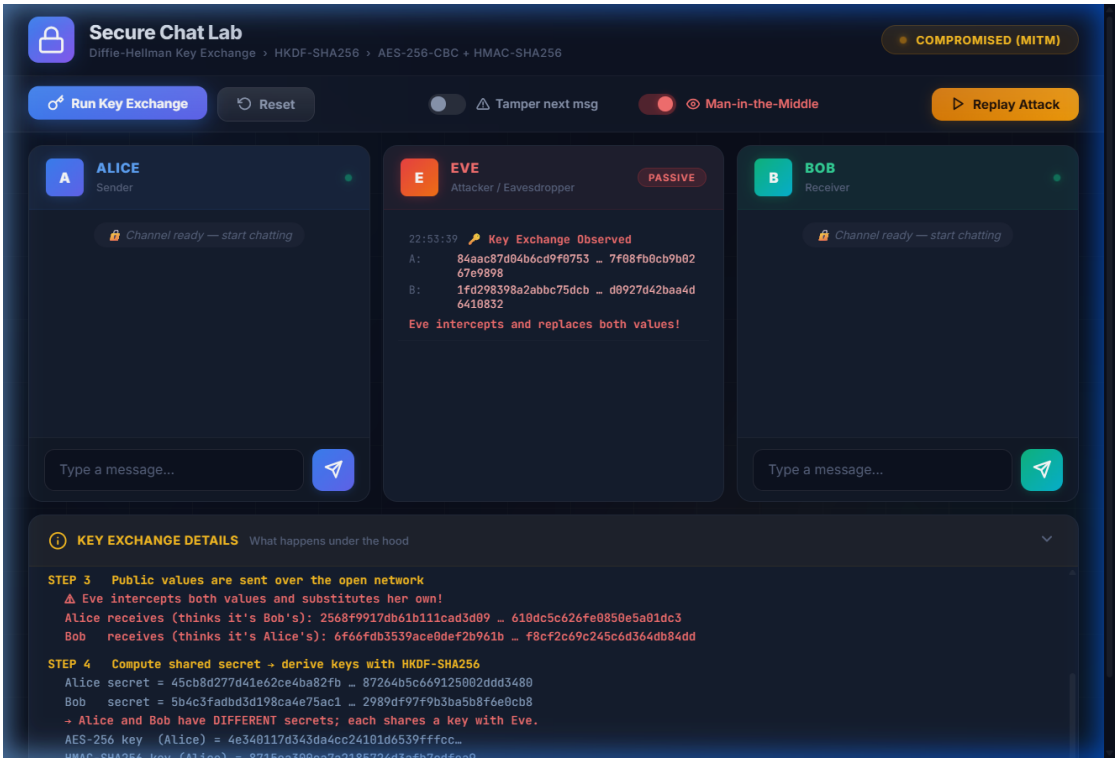


Figure 9: Eve intercepts and substitutes public keys during DH handshake.

Analysis: With MITM active, Eve intercepts Alice's public key A and Bob's public key B, substituting her own generated keys Y_{E_1} and Y_{E_2}. Alice and Bob establish separate keys with Eve while believing they are directly paired. The badge signals `COMPROMISED (MITM)`.

5.7 Figure 7: MITM Traffic Eavesdropping and Payload Manipulation

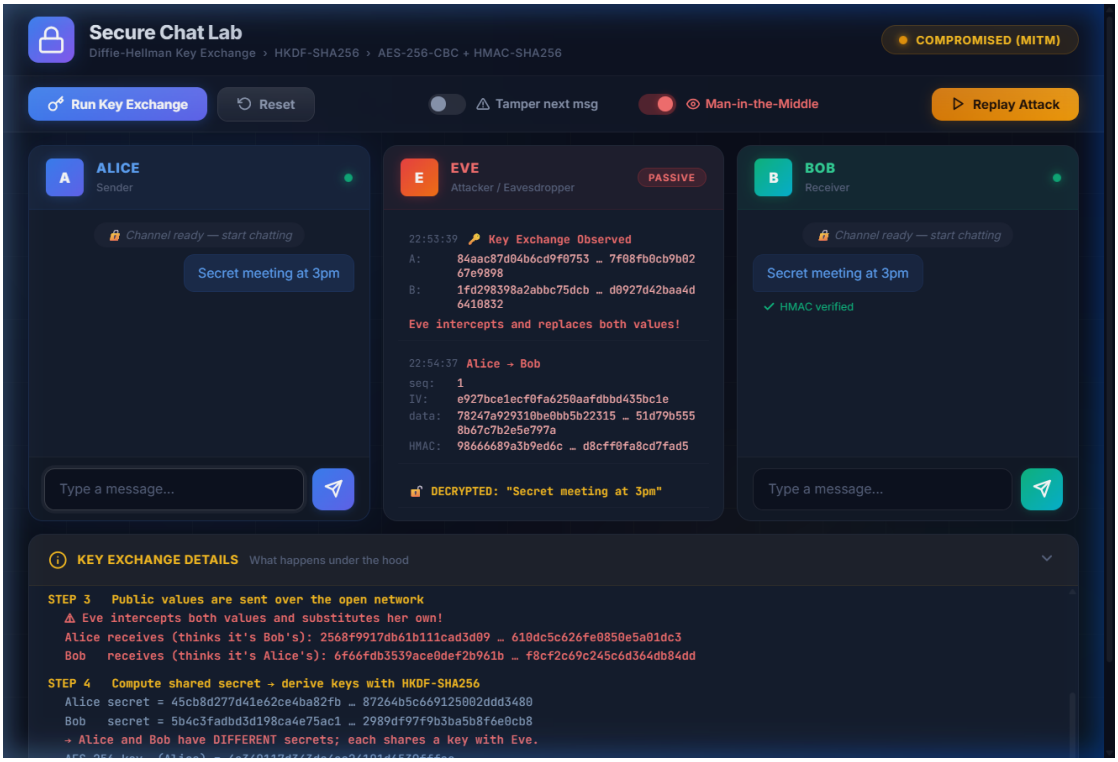


Figure 10: Eve decrypts, modifies, and re-encrypts ciphertext in transit.

Analysis: Alice sends `"Secret meeting at 3pm"`. Eve decrypts the payload using K_{AE}, reads the plaintext in clear, appends `[edited by Eve]`, re-encrypts using K_{EB}, and forwards it to Bob. Bob receives the modified text with a valid tag, proving that **unauthenticated Diffie-Hellman does not provide peer authentication**.

6 Implementation Code

The project implementation encompasses the complete cryptographic core, packet assembly, and tri-party interactive GUI. Below is the source code of `secure_chat_lab.py`:

```
import os
import struct
import time
import tkinter as tk
from tkinter import ttk

from cryptography.hazmat.primitives import hashes, hmac, padding
from cryptography.hazmat.primitives.asymmetric import dh
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
from cryptography.hazmat.primitives.kdf.hkdf import HKDF
from cryptography.exceptions import InvalidSignature

# RFC 3526 group 14 (2048-bit MODP safe prime), generator g = 2
P_HEX = (
    "FFFFFFFFFFFFFFFFC90FDAA22168C234C4C6628B80DC1CD129024E088A67CC74"
    "020BBEA63B139B22514A08798E3404DDEF9519B3CD3A431B302B0A6DF25F1437"
    "4FE1356D6D51C245E485B576625E7EC6F44C42E9A637ED6B0BFF5CB6F406B7ED"
    "EE386BFB5A899FA5AE9F24117C4B1FE649286651ECE45B3DC2007CB8A163BF05"
    "98DA48361C55D39A69163FA8FD24CF5F83655D23DCA3AD961C62F356208552BB"
    "9ED529077096966D670C354E4ABC9804F1746C08CA18217C32905E462E36CE3B"
    "E39E772C180E86039B2783A2EC07A28FB5C55DF06F4C52C9DE2BCBF695581718"
    "3995497CEA956AE515D2261898FA051015728E5A8AACAA68FFFFFFFFFFFFFFFF"
)
P = int(P_HEX, 16)
```

```python
G = 2
PARAMS = dh.DHParameterNumbers(P, G).parameters()

class Keys:
    """AES key + HMAC key derived from one shared secret via HKDF."""
    def __init__(self, shared_secret: bytes):
        okm = HKDF(hashes.SHA256(), 64, salt=None,
                   info=b"secure-chat-lab").derive(shared_secret)
        self.aes_key, self.mac_key = okm[:32], okm[32:]
        self.send_seq = 0
        self.recv_seq = 0

def encrypt_message(keys: Keys, text: str) -> bytes:
    """packet = seq(8) | iv(16) | ciphertext | hmac(32)"""
    keys.send_seq += 1
    seq = struct.pack(">Q", keys.send_seq)
    iv = os.urandom(16)
    padder = padding.PKCS7(128).padder()
    padded = padder.update(text.encode()) + padder.finalize()
    enc = Cipher(algorithms.AES(keys.aes_key), modes.CBC(iv)).encryptor()
    ct = enc.update(padded) + enc.finalize()
    h = hmac.HMAC(keys.mac_key, hashes.SHA256())
    h.update(seq + iv + ct)
    return seq + iv + ct + h.finalize()

def decrypt_message(keys: Keys, packet: bytes, check_replay=True) -> str:
    """Verify HMAC first, then replay check, then decrypt."""
    if len(packet) < 8 + 16 + 16 + 32:
        raise ValueError("packet too short")
    seq, iv, ct, tag = packet[:8], packet[8:24], packet[24:-32], packet[-32:]
    h = hmac.HMAC(keys.mac_key, hashes.SHA256())
    h.update(seq + iv + ct)
    try:
        h.verify(tag)
    except InvalidSignature:
        raise ValueError("HMAC verification FAILED - message was modified")
    n = struct.unpack(">Q", seq)[0]
    if check_replay and n <= keys.recv_seq:
        raise ValueError(f"REPLAY detected - sequence {n} already seen")
    dec = Cipher(algorithms.AES(keys.aes_key), modes.CBC(iv)).decryptor()
    padded = dec.update(ct) + dec.finalize()
    unpadder = padding.PKCS7(128).unpadder()
    text = unpadder.update(padded) + unpadder.finalize()
    if check_replay:
        keys.recv_seq = n
    return text.decode()

class Party:
    """One communication participant (Alice, Bob, or Eve's fake persona)."""
    def __init__(self, name):
        self.name = name
        self.priv = PARAMS.generate_private_key()
        self.x = self.priv.private_numbers().x
        self.y = self.priv.public_key().public_numbers().y
        self.keys = None

    def compute_shared(self, peer_y: int) -> bytes:
        peer = dh.DHPublicNumbers(peer_y, PARAMS.parameter_numbers()).public_key()
        secret = self.priv.exchange(peer)
        self.keys = Keys(secret)
        return secret
```

Listing 1: Python Cryptographic Engine and Protocol Pipeline (secure_chat_lab.py)

7 Security Discussion & Theoretical Guarantees

7.1 Why Encrypt-then-MAC (EtM)?

Classical protocols historically experimented with three generic composition paradigms:

- **MAC-then-Encrypt (MtE):** (Used in TLS 1.0–1.2). $C = E(P \parallel \text{MAC}(P))$. Vulnerable to timing side-channels and padding oracle attacks (e.g., Lucky Thirteen).

- **Encrypt-and-MAC (E&M):** (Used in SSH). $C = E(P), T = \text{MAC}(P)$. The MAC tag can leak information about the plaintext because MACs are not guaranteed to be pseudorandom.
- **Encrypt-then-MAC (EtM):** (Used in IPsec, TLS 1.3, Signal). $C = E(P), T = \text{MAC}(C)$. Proven by Krawczyk (2001) to achieve IND-CCA2 security generically under any secure symmetric cipher and PRF MAC.

7.2 Mitigating Diffie-Hellman MITM Attacks

The project's MITM simulation demonstrates that while Diffie-Hellman provides computational secrecy against passive eavesdroppers, **it does not provide peer entity authentication**. To prevent MITM in production environments:

1. **Digital Signatures (PKI):** Ephemeral DH public keys (A, B) are signed using long-term asymmetric keys (e.g., ECDSA or Ed25519) certified by an X.509 Certificate Authority, exactly as in TLS 1.3.
2. **Out-of-Band Safety Numbers:** Modern secure messaging protocols (e.g., Signal, WhatsApp) compute a cryptographic fingerprint over both public keys:

$$\text{Fingerprint} = \text{SHA-512}(A \parallel B) \tag{19}$$

   Users verify this hash through QR codes or verbal confirmation.

8 Conclusion

This project successfully designed, analyzed, and implemented a robust, end-to-end secure communication system embodying modern cryptographic standards. By pairing RFC 3526 2048-bit Diffie-Hellman key exchange with HKDF-SHA256, AES-256-CBC, and Encrypt-then-MAC HMAC-SHA256 with sequence freshness verification, the architecture achieves confidentiality, data integrity, origin authenticity, and anti-replay protection. The tri-party visual attack simulation clearly demonstrates the operational mechanics of real-world cryptographic defense against passive eavesdropping, active payload tampering, replay attacks, and man-in-the-middle vulnerabilities.

References

[1] Diffie, W., & Hellman, M. (1976). New directions in cryptography. *IEEE Transactions on Information Theory*, 22(6), 644–654.

[2] Kivinen, T., & Kojo, M. (2003). *More Modular Exponential (MODP) Diffie-Hellman groups for Internet Key Exchange (IKE)*. RFC 3526, Internet Engineering Task Force (IETF).

[3] Krawczyk, H., & Eronen, P. (2010). *HMAC-based Extract-and-Expand Key Derivation Function (HKDF)*. RFC 5869, Internet Engineering Task Force (IETF).

[4] Krawczyk, H. (2001). The order of encryption and authentication for protecting communications (or: How secure is SSL?). In *Advances in Cryptology — CRYPTO 2001* (pp. 310–331). Springer, Berlin, Heidelberg.

[5] Bellare, M., & Namprempre, C. (2000). Authenticated encryption: Relations among notions and analysis of the generic composition paradigm. In *Advances in Cryptology — ASIACRYPT 2000* (pp. 531–545). Springer, Berlin, Heidelberg.

[6] National Institute of Standards and Technology (NIST). (2001). *Advanced Encryption Standard (AES)*. Federal Information Processing Standards Publication (FIPS PUB 197).

[7] National Institute of Standards and Technology (NIST). (2001). *Recommendation for Block Cipher Modes of Operation: Methods and Techniques*. NIST Special Publication 800-38A.

[8] Krawczyk, H., Bellare, M., & Canetti, R. (1997). *HMAC: Keyed-Hashing for Message Authentication*. RFC 2104, Internet Engineering Task Force (IETF).

[9] Stallings, W. (2017). *Cryptography and Network Security: Principles and Practice* (7th ed.). Pearson Education.

[10] Rescorla, E. (2018). *The Transport Layer Security (TLS) Protocol Version 1.3*. RFC 8446, Internet Engineering Task Force (IETF).